

Exam Revision Guide: Modules 6 to 9

Every pending topic from your feedback, explained simply with examples and code. Built for a 2-hour last-minute revision.

Your 2-hour plan

Time	What to do
0:00 - 0:10	Skim this whole PDF once. Know what is coming.
0:10 - 0:40	Module 6 + 7 (SQL). Type every query in a SQL editor (SQLite / MySQL / online). Do not just read.
0:40 - 1:15	Module 8 (preprocessing). Run the Python code in Jupyter. Change values and watch the output.
1:15 - 1:45	Module 9 (statistics + plots). Run the correlation + scatter plot code on any dataset.
1:45 - 2:00	Read the cheat sheet on the last page. Rewrite the 'one-liners' from memory.

Rule for today: your feedback says the gap is **practical**, not definitions. For every topic: read 2 minutes, then type the example yourself. Typing a query and seeing its output teaches more than re-reading.

MODULE 6: SQL Basics

6.1 Query language vs programming language

A **query language** (SQL) is **declarative**: you say *what* data you want, and the database decides *how* to get it. A **programming language** (Python, Java) is **procedural**: you write step-by-step instructions (loops, conditions, functions) for *how* to do the work.

	Query language (SQL)	Programming language (Python)
Style	Declarative (what)	Procedural / general purpose (how)
Purpose	Store, retrieve, update data in a database	Build any kind of software or logic
Loops / functions	Limited	Full support
Example	SELECT name FROM students;	for s in students: print(s.name)

Exam line: SQL is a domain-specific, declarative query language used to communicate with relational databases. It is not designed for general application logic.

6.2 CHECK constraint

CHECK makes sure a column value satisfies a condition. If the condition is false, the insert or update is rejected.

```
CREATE TABLE students (  
    id      INT PRIMARY KEY,  
    name    VARCHAR(50),  
    age     INT CHECK (age >= 18),  
    score   INT CHECK (score BETWEEN 0 AND 100)  
);  
  
INSERT INTO students VALUES (1, 'Sam', 17, 80);    -- FAILS: age < 18  
INSERT INTO students VALUES (2, 'Ravi', 20, 85);    -- OK
```

6.3 Primary Key vs UNIQUE

	PRIMARY KEY	UNIQUE
Duplicates	Not allowed	Not allowed
NULL values	Not allowed	Allowed (usually one NULL in most DBs, MySQL allows several)
How many per table	Only one	Many
Purpose	Identifies each row uniquely	Stops duplicates in other columns (email, phone)

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY,    -- one PK, never NULL  
    email   VARCHAR(100) UNIQUE, -- unique but can be NULL  
    phone   VARCHAR(15) UNIQUE  -- a table can have many UNIQUE columns  
);
```

6.4 Increase score by 2 (UPDATE)

```
UPDATE students  
SET score = score + 2;    -- all rows  
  
UPDATE students  
SET score = score + 2  
WHERE id = 1;            -- only one row
```

Always add WHERE unless you truly want to change every row. Forgetting WHERE is the most common UPDATE mistake.

6.5 Names starting with 's' (LIKE)

LIKE does pattern matching. % = any number of characters, _ = exactly one character.

```
SELECT * FROM students WHERE name LIKE 's%'; -- starts with s
SELECT * FROM students WHERE name LIKE '%s'; -- ends with s
SELECT * FROM students WHERE name LIKE '%s%'; -- contains s
SELECT * FROM students WHERE name LIKE '_a%'; -- second letter is a
```

Note: in MySQL LIKE is case-insensitive by default. In PostgreSQL use ILIKE for case-insensitive, or LOWER(name) LIKE 's%'.

6.6 Total salary of employees with no last name

'No last name' means the value is **NULL**. You cannot use = NULL. You must use **IS NULL**. SUM adds the salaries.

```
SELECT SUM(salary) AS total_salary
FROM employees
WHERE last_name IS NULL;
```

If 'no last name' could also mean an empty string, use: WHERE last_name IS NULL OR last_name = ''

Key idea: NULL means unknown or missing. Comparisons with NULL (=, !=) never work. Use IS NULL / IS NOT NULL. Aggregate functions like SUM and AVG ignore NULLs.

MODULE 7: Advanced SQL

7.1 Denormalization

Normalization splits data into many tables to remove duplication. **Denormalization** does the opposite on purpose: it combines tables or duplicates some data so that reads are faster.

Why use it: fewer JOINS means faster SELECT queries. Good for reporting, dashboards, and read-heavy systems.

Trade-offs: more storage, duplicated data, and updates become harder because the same value must be changed in several places (risk of inconsistency).

```
-- Normalized: need a JOIN every time
orders(order_id, customer_id, amount)
customers(customer_id, customer_name)

-- Denormalized: name copied into orders, no JOIN needed
orders(order_id, customer_id, customer_name, amount)
```

Exam line: Denormalization deliberately adds redundancy to improve read performance at the cost of write complexity and storage.

7.2 Correlated subquery

A normal subquery runs **once** and hands its result to the outer query. A **correlated subquery** uses a column from the outer query, so it runs **once for every outer row**. The inner query depends on the outer one.

Example tables:

emp_id	name	dept	salary
1	Asha	IT	50000
2	Ben	IT	70000
3	Cara	HR	40000
4	Dev	HR	60000

Task: find employees who earn more than the average salary **of their own department**.

```
SELECT name, dept, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE dept = e.dept      -- uses e.dept from the OUTER query
);
```

How it runs: for Asha (IT), inner query computes AVG of IT = 60000, 50000 > 60000 is false. For Ben (IT), 70000 > 60000 is true. For Cara (HR), avg HR = 50000, 40000 > 50000 false. For Dev (HR), 60000 > 50000 true.

Result: Ben and Dev.

Spot it: if the inner query cannot run alone (it refers to e.dept), it is correlated.

7.3 Window functions

A window function does a calculation across a set of related rows **but keeps every row**. GROUP BY collapses rows into one per group; a window function does not. Syntax: function() OVER (PARTITION BY ... ORDER BY ...)

```
-- Average salary per dept, shown next to EVERY employee
SELECT name, dept, salary,
```

```

        AVG(salary) OVER (PARTITION BY dept) AS dept_avg
FROM employees;

-- Rank inside each department (highest salary = 1)
SELECT name, dept, salary,
       RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
FROM employees;

-- Running total
SELECT name, salary,
       SUM(salary) OVER (ORDER BY emp_id) AS running_total
FROM employees;

```

Function	What it does
ROW_NUMBER()	1,2,3,4 always unique, no ties
RANK()	Ties share a rank, then skips (1,1,3)
DENSE_RANK()	Ties share a rank, no skip (1,1,2)
LAG(col)	Value from the previous row
LEAD(col)	Value from the next row
SUM/AVG/COUNT() OVER	Aggregate without collapsing rows

GROUP BY vs window: GROUP BY dept gives 2 rows (one per dept). PARTITION BY dept gives all 4 rows, each with its dept value attached.

7.4 Foreign keys

A **foreign key** is a column that points to the **primary key of another table**. It creates the relationship and enforces **referential integrity**: you cannot insert a child row whose parent does not exist.

```

CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- Add a foreign key to an existing table
ALTER TABLE employees
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_id) REFERENCES departments(dept_id);

INSERT INTO departments VALUES (1, 'IT');
INSERT INTO employees VALUES (10, 'Asha', 1);    -- OK
INSERT INTO employees VALUES (11, 'Ben', 99);    -- ERROR: dept 99 does not exist

```

Use a JOIN to read related data:

```

SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;

```

ON DELETE options: CASCADE (delete children too), SET NULL (child FK becomes NULL), RESTRICT (block the delete).

7.5 Practice (do these now, 10 minutes)

- Create the departments and employees tables above with 4 rows each.

- Write the correlated subquery for 'above dept average' and check it matches the table.
- Write `RANK() OVER (PARTITION BY dept ORDER BY salary DESC)` and see how output changes if you remove `PARTITION BY`.
- Try inserting an employee with a `dept_id` that does not exist and read the error.

MODULE 8: Data Preprocessing

8.1 Types of data

Category	Type	Meaning	Example
Numerical	Discrete	Countable whole numbers	Number of children, cars
Numerical	Continuous	Any value in a range (measured)	Height, weight, temperature
Categorical	Nominal	Categories with NO order	Colour, city, gender
Categorical	Ordinal	Categories WITH a meaningful order	Low/Medium/High, grades, ratings
Other	Binary	Only two values	Yes/No, 0/1
Other	Time series / text	Ordered by time / free text	Daily sales, reviews

Scale levels you may be asked: **Nominal**, **Ordinal**, **Interval** (order + equal gaps, no true zero, e.g. temperature in Celsius), **Ratio** (has true zero, e.g. height, salary).

Why it matters: the data type decides the preprocessing. Categorical needs encoding. Numerical needs scaling.

8.2 Encoding categorical variables

Models need numbers, so categories must be converted.

Method	Use for	Example
Label / Ordinal encoding	Ordinal data	Low=0, Medium=1, High=2
One-hot encoding	Nominal data	Red, Blue, Green become 3 columns of 0/1

```
import pandas as pd
df = pd.DataFrame({'color': ['Red', 'Blue', 'Green', 'Red'],
                   'size': ['Low', 'High', 'Medium', 'Low']})

# One-hot for nominal
pd.get_dummies(df['color'])

# Ordinal mapping
df['size_enc'] = df['size'].map({'Low':0, 'Medium':1, 'High':2})

# sklearn versions
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
```

Do not label-encode nominal data (Red=0, Blue=1, Green=2) because the model will think Green > Red.

8.3 Scaling and standardization

Features have different ranges (age 20-60, salary 20000-200000). Distance-based and gradient-based models (KNN, SVM, logistic regression, neural nets, PCA) get dominated by the big-range feature. Scaling puts features on comparable ranges.

	Normalization (Min-Max scaling)	Standardization (Z-score)
Formula	$(x - \min) / (\max - \min)$	$(x - \text{mean}) / \text{std}$
Result range	0 to 1	Mean 0, std 1 (no fixed range)
Sensitive to outliers?	Yes, very	Less
Use when	Bounded range needed, no strong outliers, neural nets / images	Data roughly bell-shaped, outliers present, SVM, regression, PCA

Worked example: ages 20, 30, 40. Min-max: 0, 0.5, 1. Standardization: mean = 30, std = about 8.16, so $z = \frac{x - \text{mean}}{\text{std}}$ = -1.22, 0, +1.22.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import numpy as np
X = np.array([[20],[30],[40]])

MinMaxScaler().fit_transform(X)    # 0, 0.5, 1
StandardScaler().fit_transform(X)  # -1.22, 0, 1.22
```

Tree-based models (Decision Tree, Random Forest) do not need scaling.

8.4 Data preprocessing pipeline

A **pipeline** chains preprocessing steps and the model into one object so the same steps run in the same order for training and new data. Typical steps: handle missing values, encode categoricals, scale numerics, then the model.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

num_cols = ['age', 'salary']
cat_cols = ['city']

num_pipe = Pipeline([('impute', SimpleImputer(strategy='median')),
                     ('scale', StandardScaler())])
cat_pipe = Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                     ('onehot', OneHotEncoder(handle_unknown='ignore'))])

pre = ColumnTransformer([('num', num_pipe, num_cols),
                        ('cat', cat_pipe, cat_cols)])

model = Pipeline([('pre', pre), ('clf', LogisticRegression())])
model.fit(X_train, y_train)    # learns from TRAIN only
print(model.score(X_test, y_test))
```

Benefits: reproducible, cleaner code, fewer mistakes, and it **prevents data leakage** automatically.

8.5 Data leakage

Data leakage is when information from outside the training data (from the test set or from the future) influences the model during training. The model looks great in testing but fails in real life.

Two common types:

- **Train-test contamination:** fitting a scaler / imputer / encoder on the **full dataset** before splitting. The test set's mean and std leak into training.
- **Target leakage:** a feature that would not be known at prediction time or directly reveals the answer (e.g. 'loan_repaid' used to predict 'loan_default').

```
# WRONG (leakage)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)    # sees test data
X_train, X_test = train_test_split(X_scaled)

# RIGHT
```

```
X_train, X_test = train_test_split(X)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)    # fit on train ONLY
X_test_s  = scaler.transform(X_test)        # only transform on test
```

Golden rule: **fit on train, transform on test**. Split first, preprocess after. Pipelines do this for you.

MODULE 9: Statistics and Visualization

9.1 Kurtosis

Kurtosis measures the **tailedness** of a distribution: how heavy the tails are, meaning how many extreme values (outliers) you get compared with a normal distribution. (It is not simply 'peakedness'.)

Type	Excess kurtosis	Shape	Meaning
Mesokurtic	about 0 (raw = 3)	Normal bell curve	Normal tails
Leptokurtic	> 0	Sharp peak, fat tails	More outliers
Platykurtic	< 0	Flat, thin tails	Fewer outliers

Do not confuse with **skewness**, which measures asymmetry (left or right tilt). Skewness = lopsidedness; kurtosis = tail weight.

```
from scipy.stats import kurtosis, skew
kurtosis(df['height']) # excess kurtosis (normal = 0)
skew(df['height'])
df['height'].kurt() # pandas, also excess kurtosis
```

9.2 Covariance

Covariance measures how two variables **change together**.

- **Positive:** when one goes up, the other tends to go up (height and weight).
- **Negative:** when one goes up, the other tends to go down (speed and travel time).
- **Near zero:** no linear relationship.

Formula: $\text{cov}(X,Y) = \text{sum of } (x - \text{mean}_x)(y - \text{mean}_y) / (n - 1)$

Problem: its value depends on the units (cm vs metres), so the size is hard to interpret. That is why we use **correlation**, which is covariance scaled to be between -1 and +1.

	Covariance	Correlation (Pearson)
Range	-infinity to +infinity	-1 to +1
Units	Has units, scale-dependent	Unitless, scale-free
Tells	Direction only	Direction and strength
Formula	$\text{cov}(X,Y)$	$\text{cov}(X,Y) / (\text{std}_x * \text{std}_y)$

Interpreting correlation: +1 perfect positive, 0 none, -1 perfect negative. Roughly: |r| above 0.7 strong, 0.3 to 0.7 moderate, below 0.3 weak. **Correlation does not imply causation.**

```
import numpy as np
df['height'].cov(df['weight'])
np.cov(df['height'], df['weight']) # covariance matrix
```

9.3 Which plot for students' heights?

Height is **continuous numerical** data, and you want to see how the values are distributed. Answer: **Histogram** (or a KDE / density plot). A **box plot** is also good for spread and outliers.

Goal	Data	Best plot
Distribution of one numeric variable (heights)	Continuous	Histogram , KDE, box plot
Relationship between two numeric variables	Numeric vs numeric	Scatter plot

Goal	Data	Best plot
Compare categories	Categorical vs number	Bar chart, box plot
Proportion of a whole	Categorical	Pie chart (few categories) or bar
Trend over time	Time series	Line plot
Correlations among many features	Many numeric	Heatmap of correlation matrix

Why not a bar chart? Bar charts are for categories. Heights are continuous, so a histogram groups them into bins and shows the shape (normal, skewed, etc.).

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.hist(df['height'], bins=10, edgecolor='black')
plt.xlabel('Height (cm)'); plt.ylabel('Number of students')
plt.title('Distribution of Student Heights')
plt.show()

sns.boxplot(x=df['height'])      # spread + outliers
```

9.4 Practical: correlation + visualization (the part you missed)

Task: find the correlation between two features and visualize it. Steps: load data, pick two numeric columns, compute correlation, draw a scatter plot, interpret.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data.csv')          # or build a tiny one:
# df = pd.DataFrame({'height':[150,160,165,170,180],
#                    'weight':[50,58,62,68,80]})

# 1. Correlation between two features
r = df['height'].corr(df['weight'])    # Pearson by default
print('Correlation:', r)

# 2. Scatter plot
plt.scatter(df['height'], df['weight'])
plt.xlabel('Height'); plt.ylabel('Weight')
plt.title(f'Height vs Weight (r = {r:.2f})')
plt.show()

# 3. Optional: line of best fit + heatmap of all correlations
sns.regplot(x='height', y='weight', data=df)
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```

Interpretation template: 'The correlation between height and weight is 0.98, which is a strong positive correlation. As height increases, weight tends to increase. This does not prove causation.'

If asked to do this in the exam: three lines are enough: `.corr()`, `plt.scatter()`, and one sentence of interpretation.

Last-Page Cheat Sheet (read this in the final 15 minutes)

Topic	One-liner
Query vs programming language	SQL = declarative (what). Python = procedural (how).
CHECK	Rejects rows that fail a condition: CHECK (age >= 18).
PK vs UNIQUE	PK: one per table, no NULL. UNIQUE: many per table, NULL allowed.
Increase score by 2	UPDATE t SET score = score + 2 WHERE ...;
Names starting with s	WHERE name LIKE 's%'
No last name	WHERE last_name IS NULL (never = NULL)
Denormalization	Add redundancy on purpose for faster reads; costs storage + update complexity.
Correlated subquery	Inner query uses outer row's column, runs once per outer row.
Window function	Calculates over related rows, keeps all rows. OVER (PARTITION BY ... ORDER BY ...).
Foreign key	Column referencing another table's PK; enforces referential integrity.
Data types	Numeric (discrete, continuous); Categorical (nominal, ordinal).
Scaling	Min-max: $(x - \min) / (\max - \min)$, 0 to 1.
Standardization	$(x - \text{mean}) / \text{std}$, mean 0, std 1.
Pipeline	Chained preprocessing + model; same steps for train and test.
Data leakage	Test/future info leaking into training. Fit on train, transform on test.
Kurtosis	Tail heaviness (outliers). Normal excess = 0.
Covariance	How two variables move together; direction only, scale-dependent.
Correlation	Covariance scaled to -1..+1; direction + strength.
Heights plot	Histogram (or box plot / KDE).
Two features relation	Scatter plot + <code>df['a'].corr(df['b'])</code> .

Common mistakes to avoid

- Writing = NULL instead of IS NULL.
- UPDATE without WHERE.
- Confusing GROUP BY (collapses rows) with window functions (keeps rows).
- Scaling before train/test split (leakage).
- Label-encoding nominal data.
- Using a bar chart for continuous data like height.
- Saying correlation proves causation.

If you have spare minutes

Answer each of the earlier 'Practice' items without looking. If you can write the query or code from a blank screen, you are ready. If you get stuck, go back to that section only.